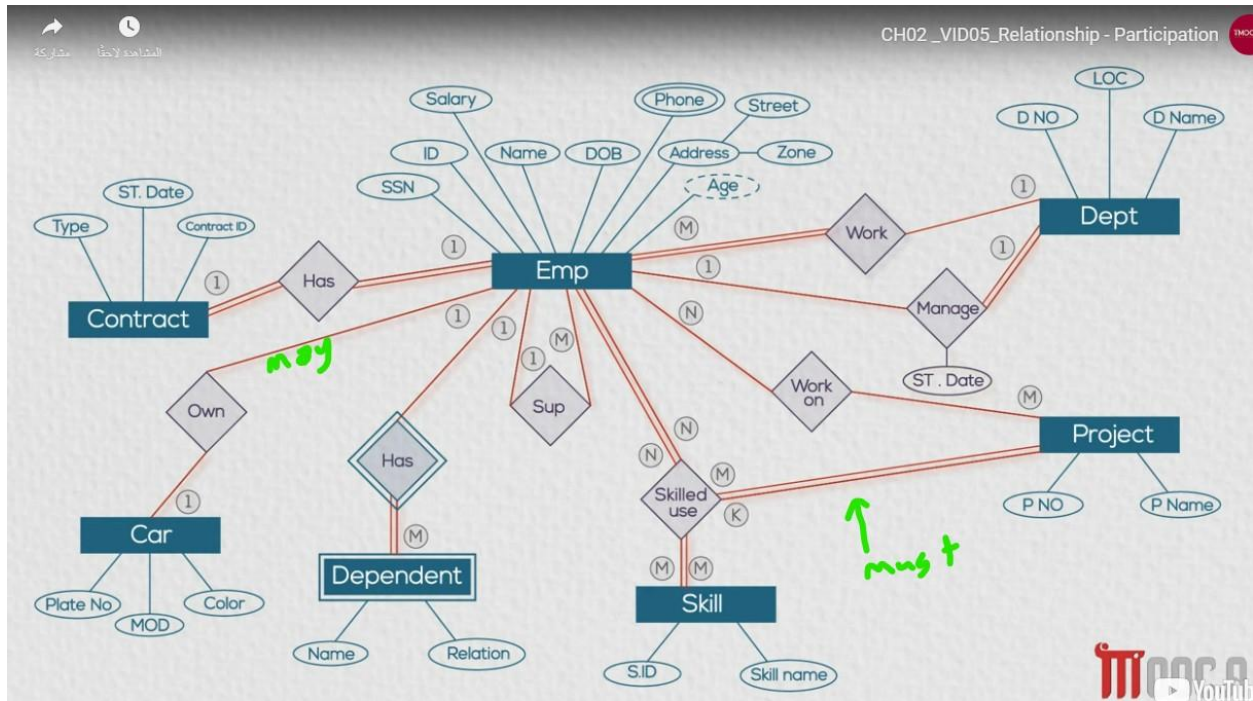




الانترينيويس :

- عادي
- درايڤيد يعني بتيڭي من انترينيويس ثانيه (داشيد)
- مالنفيالود يعني ليها اكتر من نوع (دايرتين جوا بعض)

الكيز :

- كانديديت كي : دا اي كي تقدري بيه تعرف ريكورد معين
 - برايماري كي : دا الكي اللي انت مختاره من ضمن الكانديديت كيز
- عشان يبقا الايدينتيفايير بتاعك زي مثلا رقم القومي و اي دي الطالب ف دول اثنين تعرف تعرف بيهم الطالب وبياناته بس انت مثلا ادم في الكليه ف مش هتختار الرقم القومي لا هتختار الاي دي تعرف بيه الطالب فهمت

- الفورين كي : دا بيستخدم عشان تربط اتنين تيبول ببعض بمعنى انت معاك تيبول للطالب و تيبول للكورسات ف كل طالب هياخد عدد من الكورسات هتربطهم ازاي عن طريق انك تاخذ البرايماري كي بتاع الطالب وتروح تحطه ك فورين كي في الكورسات عشان يبقا كل كورس جمبه رقم الطالب

الانتيتي:

- التيبول بتاعنا او الكائن او الحاجة اللي بنتعامل معاها و بيكون ليها اتريبوتس
- الويك منها بتبقا علي شكل مستطيل وبراها مستطيل ثاني فارغ
- واي ريليشن بتتوصل بيها بتبقا هي كمان ويك ونفس الحوار سمبوكسه براها سمبوكسه فارغه

الكارديناليتي (عدد العلاقات):

- One-to-One (1:1)

- كل كائن في الجدول الأول يرتبط بكائن واحد فقط في الجدول الثاني.

- مثال: كل موظف له بطاقة هوية واحدة فقط.

- One-to-Many (1:N)

- كائن واحد في الجدول الأول ممكن يرتبط بعدة كائنات في الجدول الثاني.
- مثال: قسم واحد فيه أكثر من موظف.

- Many-to-Many (M:N)

- أكثر من كائن في الجدول الأول يرتبط بأكثر من كائن في الجدول الثاني.
- مثال: طالب ممكن يسجل في أكثر من مادة، والمادة ممكن يدرسها أكثر من طالب.

Participation (المشاركة / الإلزامية):

هي اللي بتحدد إذا كان المشاركة في العلاقة إلزامية ولا اختيارية لكل كيان.
الأنواع:

- Total Participation مشاركة كلية : (MUST =)

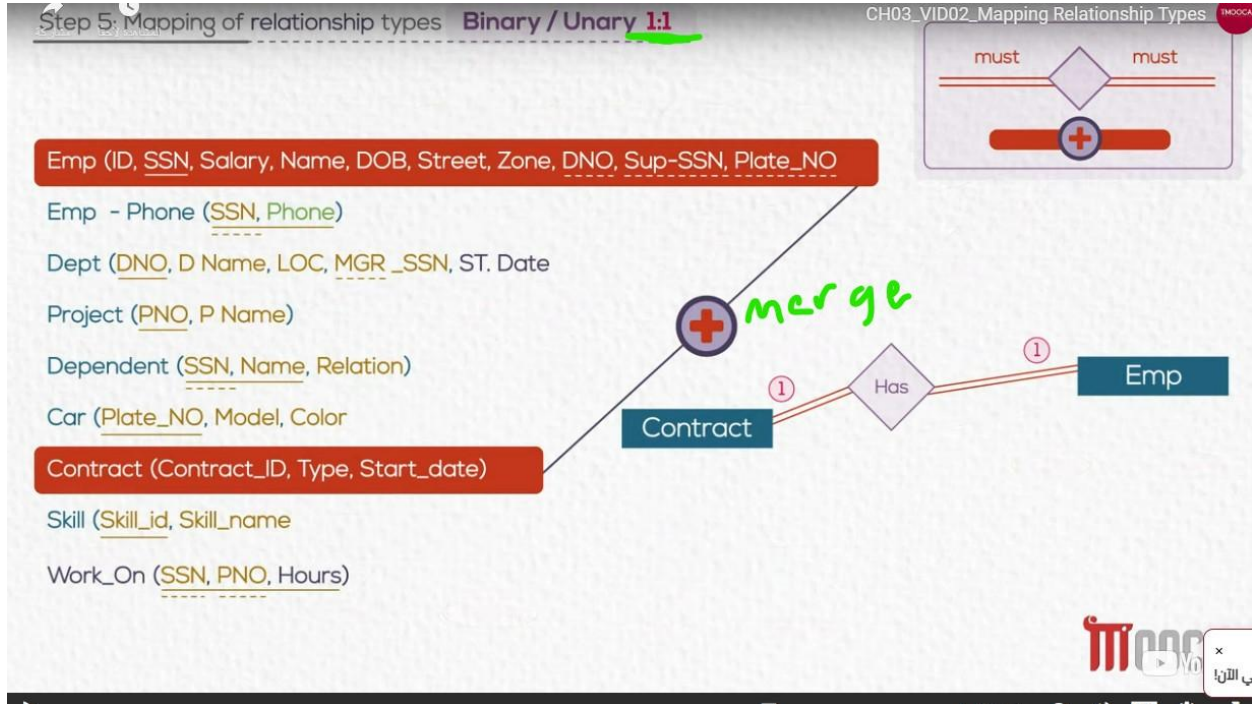
- يعني كل كائن في الكيان لازم يشارك في العلاقة.
- نرسم لها غالبًا بخط مزدوج في ERD.
- مثال: كل طالب لازم يكون مسجل في قسم.

- Partial Participation مشاركة جزئية : (MAY -)

- يعني بعض الكائنات بس ممكن يشاركوا في العلاقة (مش إلزامي).
- نرسم لها بخط مفرد.
- مثال: مش كل موظف لازم يكون مدير مشروع.

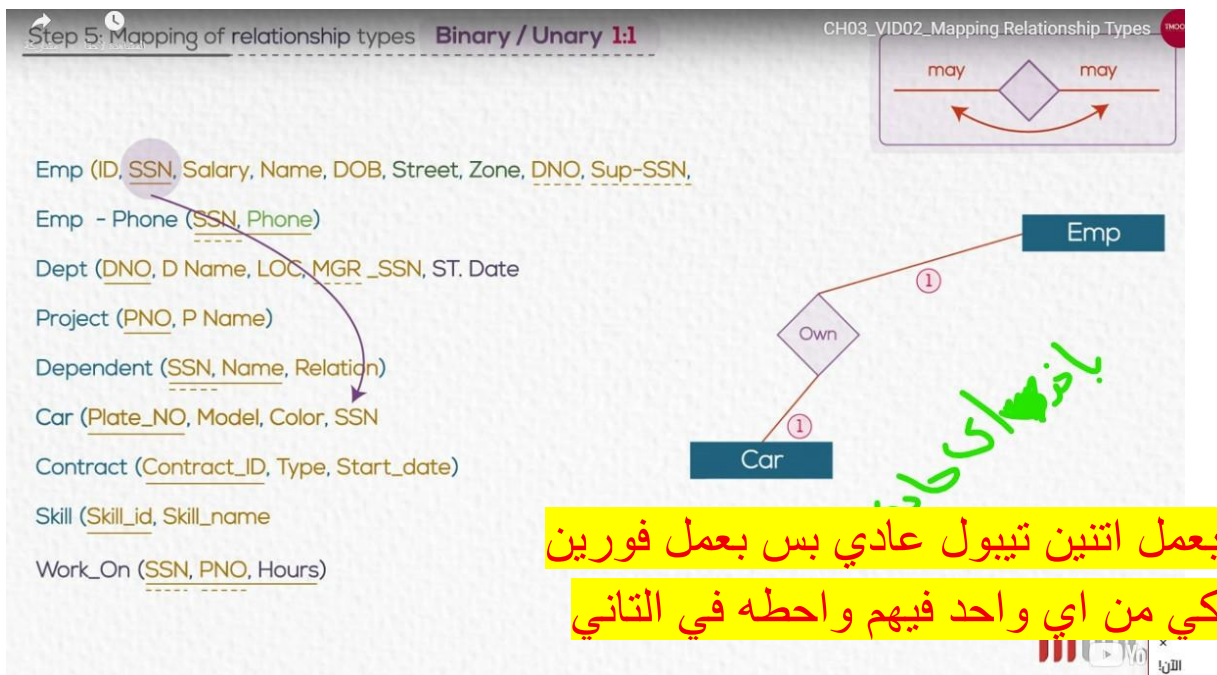
المابينج مابين اللوجيك دياگرامز او ال ER DIAGRAMS :

- لو كانت 1:1 و ماست ماست



بجمع الاتنين تيبول مع بعض في تيبول واحد

- لو ماي ماي



- طب لو اخر حاجة ماي ماست

Step 5: Mapping of relationship types Binary / Unary 1:1 CH03_VID02_Mapping Relationship Types

Emp (ID, SSN, Salary, Name, DOB, Street, Zone, DNO, Sup-SSN)
 Emp - Phone (SSN, Phone)
 Dept (DNO, D Name, LOC, MGR_SSN)
 Project (PNO, P Name)
 Dependent (SSN, Name, Relation)
 Car (Plate_NO, Model, Color)
 Contract (Contract_ID, Type, Start_date)
 Skill (Skill_id, Skill_name)
 Work_On (SSN, PNO, Hours)

Diagram showing a 1:1 relationship between Emp and Dept via a relationship type 'Manage'. The relationship type 'Manage' has a cardinality of 1 on both sides. A handwritten note in green says 'جز من اصله must'.

بتعمل 2 تيبول لكل واحد و تاخذ فورين كي من اللي ناحيته ماي وتحطه
 في اللي ناحيته ماست

- ال M:1 اي كانت ايه هتعمل تيبول جديد وتخلي الفورين من ال 1 ل
 M

Step 3: Mapping of relationship types Binary / Unary 1:N CH03_VID02_Mapping Relationship Types Add FK to N-side table

Emp (ID, SSN, Salary, Name, DOB, Street, Zone, DNO, Sup-SSN)
 Emp - Phone (SSN, Phone)
 Dept (DNO, D Name, LOC)
 Project (PNO, P Name)
 Dependent (SSN, Name, Relation)
 Car (Plate_NO, Model, Color)
 Contract (Contract_ID, Type, Start_date)
 Skill (Skill_id, Skill_name)

Diagram showing a 1:M relationship between Emp and Dependent via a relationship type 'Has'. The relationship type 'Has' has a cardinality of 1 on the Emp side and M on the Dependent side. A handwritten note in green says 'ا م'.

- لو N:M

Step 4: Mapping of relationship types Binary / Unary M:N

CH03_VID02_Mapping Relationship Types

Add FKs to the new table for both parent tables

Emp (ID, SSN, Salary, Name, DOB, Street, Zone, DNO, Sup-SSN)

Emp - Phone (SSN, Phone)

Dept (DNO, D Name, LOC)

Project (PNO, P Name)

Dependent (SSN, Name, Relation)

Car (Plate_NO, Model, Color)

Contract (Contract_ID, Type, Start_date)

Skill (Skill_id, Skill_name)

Work_On (SSN, PNO, Hours)

Work on

Hours

Emp

Project

بھل جدول جریر

بتعمل تیبول جدید و تحط فیہ الاتنین کیز بتوع الاتنین انتیتی و تخلیه کومبوسید بر ایما ری کی (یعنی بر ایما ری کی ممکن من 2 کیز) و کل کی لوحده هیبقا فورین للتیبول الاصلی بتاعه

Emp- Contract (ID, SSN, Salary, Name, DOB, Street, Zone, DNO, Sup-SSN, Plate_NO, Contract_ID, Type, Start_date)

Emp - Phone (SSN, Phone)

Dept (DNO, D Name, LOC, MGR_SSN, ST. Date)

Project (PNO, P Name)

Dependent (SSN, Name, Relation)

Car (Plate_NO, Model, Color)

Skill (Skill_id, Skill_name)

Work_On (SSN, PNO, Hours)

Skills Used (SSN, PNO, Skill_id)

Step 1: Mapping of regular entity types

Step 2: Mapping of weak entity types

Step 3: Mapping of Binary / Unary 1:M relationship types

Add FK to N-side table

Step 4: Mapping of Binary / Unary M:N relationship types

Add FKs to the new table for both parent tables

Step 5: Mapping of Binary / Unary 1:1 relationship types

Step 6: Mapping of ternary relationship types

Add FKs to the new table for all parent tables

النورماليزيشن :

النورماليزيشن دا يصحبي انك تنورماليز الحاجة بمعني انك تعيد صياغه الاسكيمه بتاعتك عشان تمشي علي حاجات معينه تظبط بيها ستر اكتوبر الداتا بيز و التوزيع بيقا صحيح عشان يقلل مشاكل زي :

- الريدانانس او ان الداتا بتتكرر وفي حاجة تانيه بس مش

- تجنب الانوماليز:

فيه 3 مشاكل أساسية بيحلها التطبيع:

• Insertion Anomaly:

○ مش قادر تضيف معلومة جديدة إلا لما تدخل معلومة تانية مالهاش لازمة.

مثال: عايز تضيف قسم جديد بس لازم تدخل طالب معاه لأنهم في نفس الجدول.

• Update Anomaly:

○ لو عندك معلومة مكررة في أكثر من مكان، وتغيرتها في مكان وما غيرتهاش في الباقي → يحصل تعارض.

• Deletion Anomaly:

○ لو مسحت سجل ممكن يضيع معاه بيانات مهمة.

مثال: لو مسحت آخر طالب في قسم → القسم كله يختفي.

- تحسين الكفاءة: (Efficiency)

- لما البيانات تكون منظمة، الاستعلامات (Queries) تبقى أسرع وأبسط.

- المرونة: (Flexibility)

- يسهل تعديل بنية قاعدة البيانات بعد كده لو احتاجنا.

1-ال ONF :

The video shows a table titled 'School Example' with the following attributes: Stud_ID (primary key), Name, Location, Tel, Level, Level_Mgr, Subject, Subj - Desc, and G. The table is highlighted with a green box. A text box above it says 'place it in a new table carrying the PK as a FK'. To the right, a legend indicates that 'Multivalued Attribute' (marked with a red X), 'Repeating group' (marked with a red X), and 'Composite attribute' (marked with a red X) are not allowed in 1NF.

Below the original table, the text '(1NF) first normal form' is written. Three new tables are shown, each with a primary key (Stud_ID) and a foreign key (Stud_ID) from the original table:

Stud_ID	Name	Location	Level	Level_Mgr
---------	------	----------	-------	-----------

Stud_ID	Tel
---------	-----

Stud_ID	Subject	Subj - Desc	G
---------	---------	-------------	---

Green arrows indicate the mapping from the original table's attributes to the new tables: Tel to the second table, and Subject, Subj - Desc, and G to the third table.

دا لازم يتحقق عندك في الجداول كلها 3 شروط عشان تبقى فيه ودا غالبا
الحاله اللي هتبقا فيها اول ما تعمل الاسكيمة
- ان يبقى عندك مالتى فايلود اتريبوت

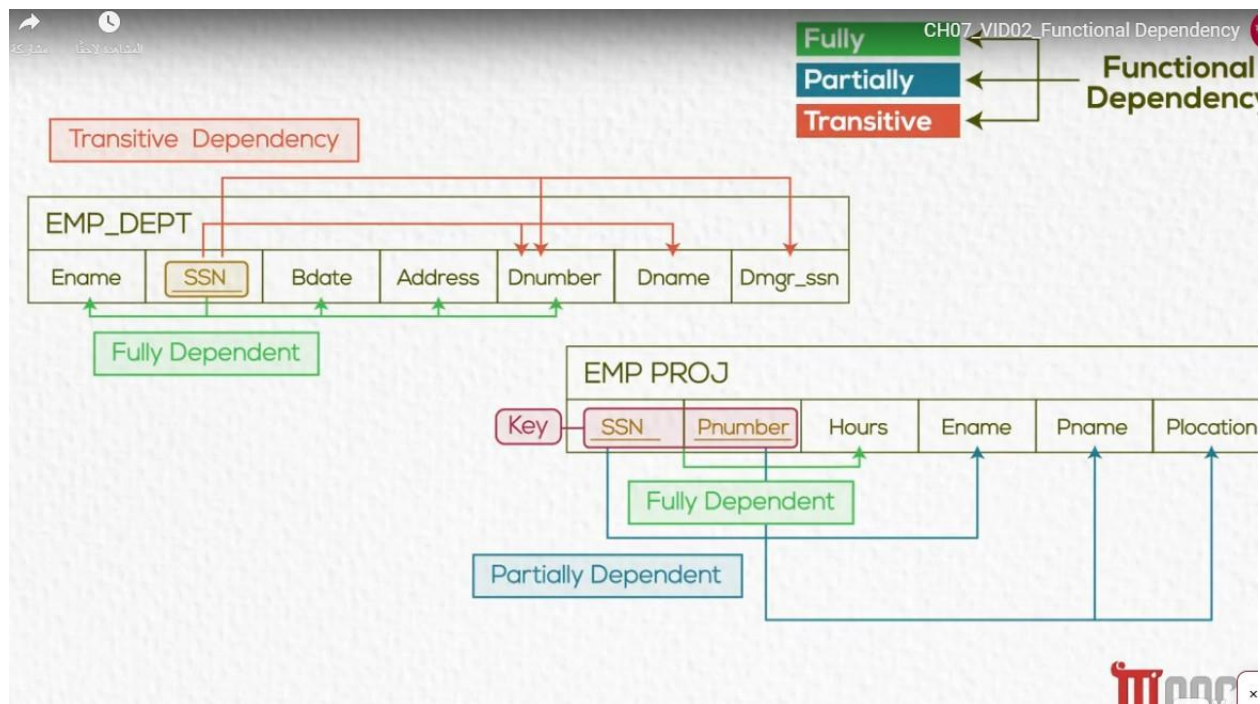
- و ريبيتينج جروب ودي بتبقا مجموعه اتريبوتس بتتكرر ومرتبطين ببعض

- كومبوزيت اتريبوت

عشان تحوله ل 1NF محتاج انك تفصل كل حاجة من دي في تيبول لوحدها وتربط ب فورين كي

1NF : زي ما قولنا بنشيل الحاجات اللي من 0NF

قبل ما نروح ل 2NF و 3NF في حاجة لازم نشرحها وهي الفانكشيونال ديبيندانس

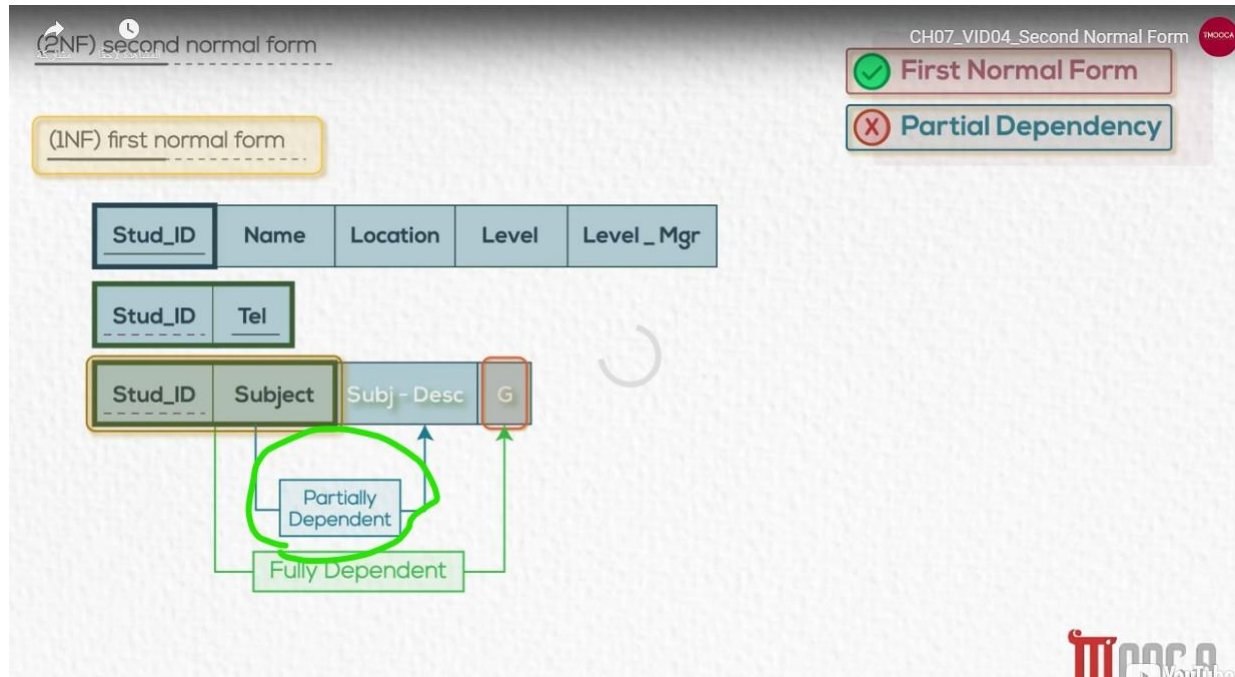


زي ما انت شايف هي بتتقسم ل 3 حاجات :

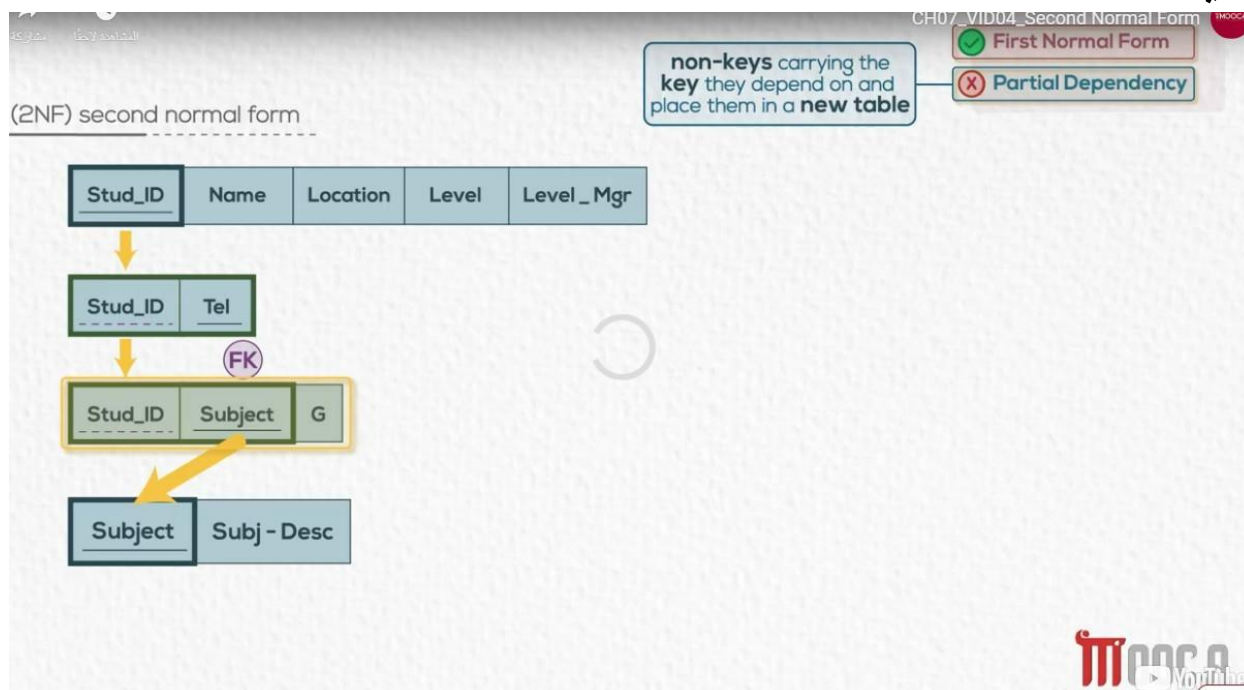
- فولي : ودي بيبقا الاتريبيوت عاديه معتمده اعتماد كلي علي البرايماري كي هتلاقيها بالاخضر
- البارشالي: دي بتبقا بتعتمد اعتماد كلي علي جزء من البرايماري مش كله يعني البرايماري هو كومبوزيت كي ف هي بتعتمد علي اتريبيوت منه لونها ازرق زي مثلا الجدول الثاني فيه PNAME معتمد علي بس ال PNUMBER مش علي البرايماري كي كله
- الترانسيتيف : هي بقا بيبقا فيها اتريبيوت عاديه معتمده علي اتريبيوت عاديه ثانيه اللي هي اصلا معتمده علي البرايماري كي ف كده الاتريبيوت الاول معتمده بشكل مش مباشر علي البرايماري كي وهي باللون الاحمر زي مثلا الديبارتمينت نيم اكيد هتعتمد علي رقم الديبارتمينت اللي هو في الاساس معتمد علي الرقم القومي بتاع الشخص

:2NF

دابقا عشان تبقا فيه لازم تحقق انك تكون في ال 1NF اصلا و يكون عندك بارشيال ديبيندانسي



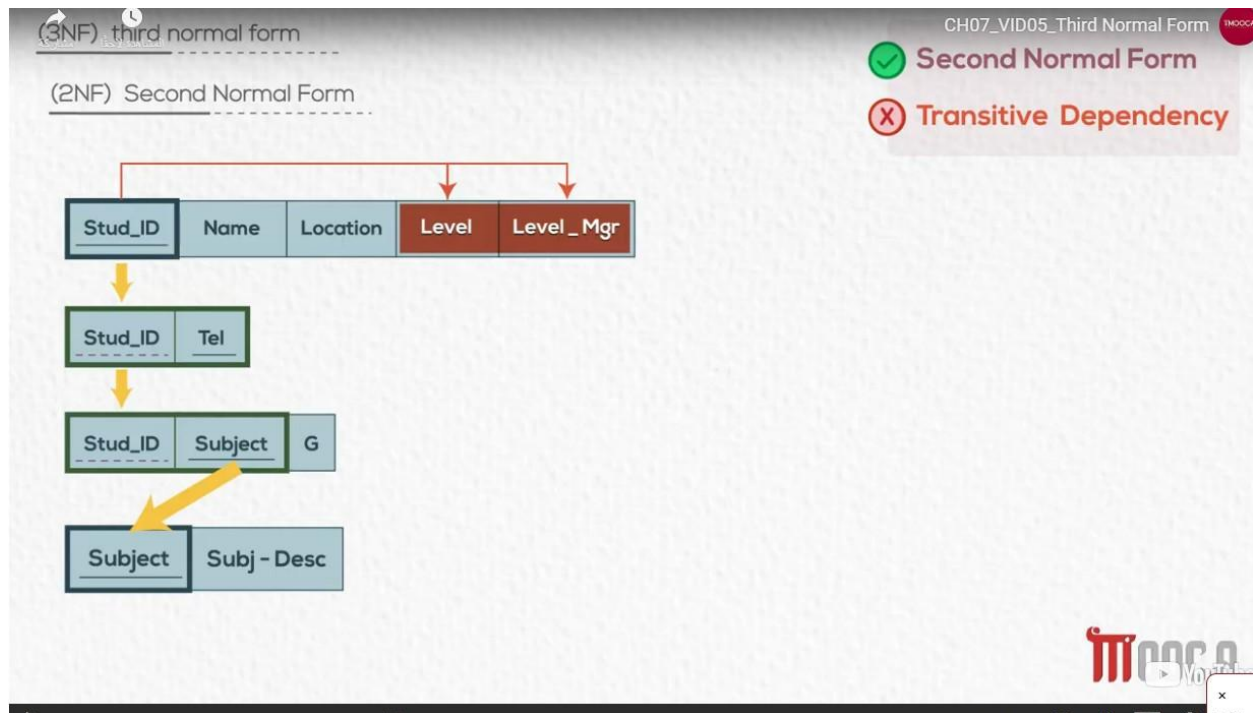
زي هنا



بتحله ازاي بقا عشان توصله لل 2NF بطلع الحاجة اللي معتمده علي بعض برا في جدول لوحدها و تسيب الكي ويبقا فورين بيشاور علي اللي في الجدول الجديد

:3NF

عشان توصله لازم يبقا معاك ال 2NF الاول ويكون عندك ترانسيتيف ديبيندانس



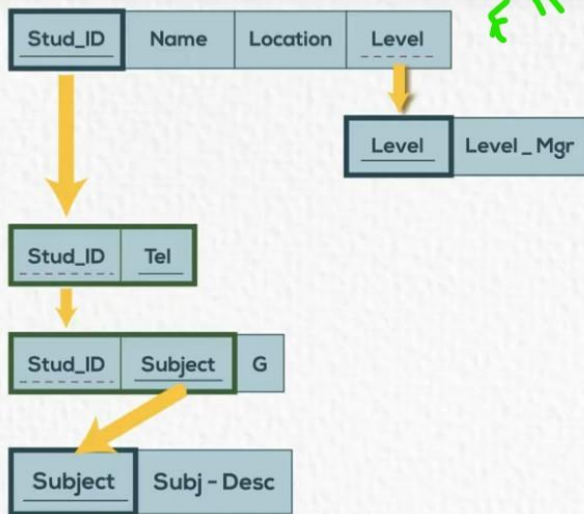
وحله برادو انك بتطلع اللي معتمد عليه و تقلبه فورين في الجدول الاولاني و تخليهم الاتنين في جدول لوحدهم

(3NF) third normal form

CH07_VID05_Third Normal Form

✓ Second Normal Form

✗ Transitive Dependency



1NF first normal form

✗ Multivalued Attribute

✗ Repeating group

✗ Composite attribute

place it in a new table carrying the PK as a FK

subparts each in a column when necessary

2NF second normal form

✓ First Normal Form

✗ Partial Dependency

non-keys carrying the key they depend on and place them in a new table

3NF third normal form

✓ Second Normal Form

✗ Transitive Dependency

non-key attributes carrying the non-key attribute they depend on and place them in a new table

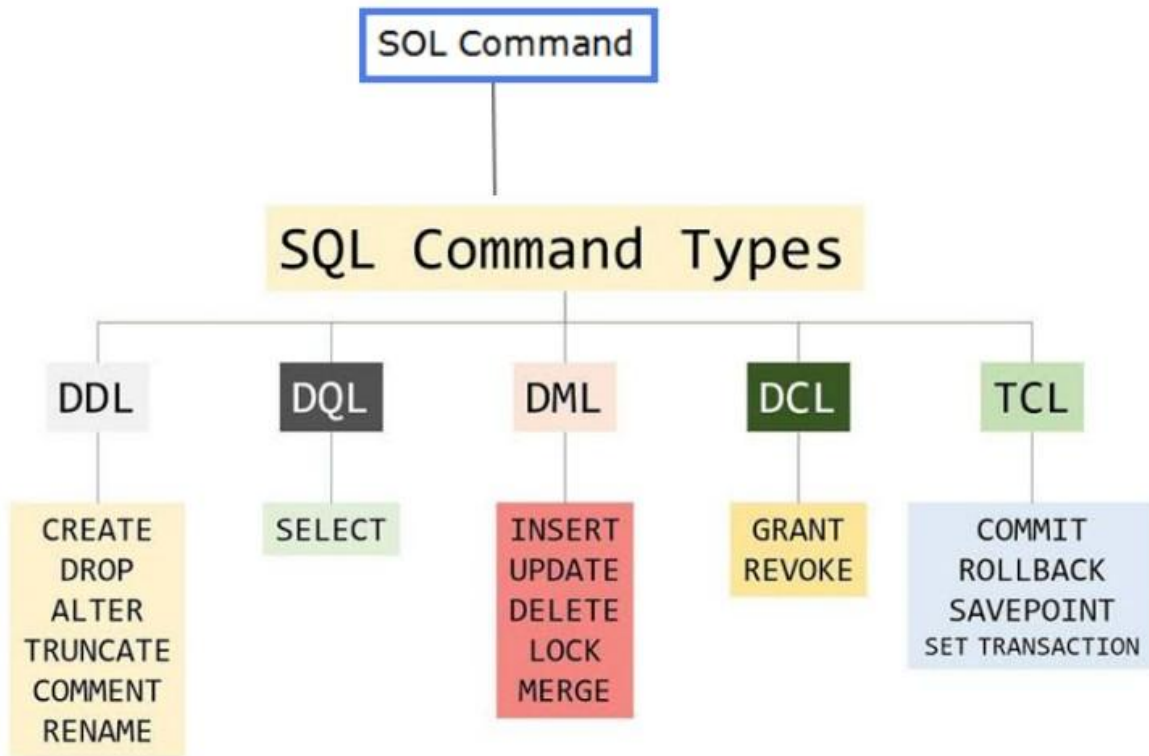


دا ملخص لکله

SQL

Types of SQL Commands

There are five types of SQL commands: DDL, DML, DCL, TCL, and DQL.



1. Data Definition Language (DDL)

- DDL changes the structure of the table like creating a table, deleting a table, altering a table, etc.
- All the command of DDL are auto-committed that means it permanently save all the changes in the database.

بتسمع علطول في الداتا بيز دي انت بتعدل في الداتا بيز ستر اكتبش يعني
في الاتريبيوتس و كده لسه محطتش داتا في التيبولز يعني

a. **CREATE** It is used to create a new table in the database.

Syntax:

1. CREATE TABLE TABLE_NAME (COLUMN_NAME DATATYPES[,....]);

Example:

1. CREATE TABLE EMPLOYEE(Name VARCHAR2(20), Email VARCHAR2(100), DOB DATE);

b. **DROP:** It is used to delete both the structure and record stored in the table.

Syntax

1. DROP TABLE ;

Example

1. DROP TABLE EMPLOYEE;

CREATE TABLE ORDERS (
OID INT NOT NULL, THEDATE DATE NOT
NULL, CID INT NOT NULL, AMOUNT INT NOT
NULL, PRIMARY KEY (OID), FOREIGN KEY (CID)
REFERENCES CUSTOMERS (ID)
)

c. **ALTER:** It is used to alter the structure of the database. This change could be either to modify the characteristics of an existing attribute or probably to add a new attribute.

Syntax:

To add a new column in the table

1. ALTER TABLE table_name ADD column_name COLUMN-definition;

To modify existing column in the table:

1. ALTER TABLE MODIFY(COLUMN DEFINITION....);

EXAMPLE

1. ALTER TABLE STU_DETAILS ADD(ADDRESS VARCHAR2(20));
2. ALTER TABLE STU_DETAILS MODIFY (NAME VARCHAR2(20));

d. **TRUNCATE:** It is used to delete all the rows from the table and free the space containing the table.

ALTER TABLE ORDERS ADD PRIMARY KEY(OID),
ADD FOREIGN KEY (CID) REFERENCES
CUSTOMERS (ID)

d. **TRUNCATE:** It is used to delete all the rows from the table and free the space containing the table.

Syntax:

1. TRUNCATE TABLE table_name;

Example:

1. TRUNCATE TABLE EMPLOYEE;

2. Data Manipulation Language

- DML commands are used to modify the database. It is responsible for all form of changes in the database.
- The command of DML is not auto-committed that means it can't permanently save all the changes in the database.
- They can be rollback.

Here are some commands that come under DML:

- INSERT
- UPDATE
- DELETE

بالتحقق
ال ACID

دي معناها انك عملت الداتا بيز ستراكتر و دخلت اوريدي داتا او عايز
تدخل او تعدل او تمسح داتا يعني اللعب في الداتا مش الاستراكتر ودي
معناها انها مش بتتسيف في الداتا بيز علطول عشان تحقق ال ACID
هنعرف يعني ايه تحت بس عشان تحافظ علي الداتا و المعاملات لو
حصلت مشكله اثناء اي عمله متبوظش الداتا بيز

a. **INSERT:** The INSERT statement is a SQL query. It is used to insert data into the row of a table.

Syntax:

1. INSERT INTO TABLE_NAME
2. (col1, col2, col3,... col N)
3. VALUES (value1, value2, value3, valueN);

Or

1. INSERT INTO TABLE_NAME
2. VALUES (value1, value2, value3, valueN);

For example:

1. INSERT INTO javatpoint (Author, Subject) VALUES ("Sonoo", "DBMS");

ممکن متکتنش القوس الی قبل فالیوز بس تبقا کاتبهم بالترتیب

b. **UPDATE:** This command is used to update or modify the value of a column in the table.

Syntax:

1. UPDATE table_name SET [column_name1= value1,...column_nameN = valueN] [WHERE CONDITION]

For example:

1. UPDATE students
2. SET User_Name = 'Sonoo'
3. WHERE Student_Id = '3'

c. **DELETE:** It is used to remove one or more row from a table.

Syntax:

1. DELETE FROM table_name [WHERE condition];

For example:

1. DELETE FROM javatpoint
2. WHERE Author="Sonoo";

الفرق بين ديليت وترانكيت ودروب

- بص ياباشا انت دروب خلينا عارفين انها بتمسح الجدول كله خالص كانه مش موجود يعني هيدي ايرور لو اكسسته ماشي ودا اخطرهم
- الترانكيت بقا دي بتمسح الداتا بس اللي في التيبول بس بتمسحهم كلهم مرة واحدة اسرع من الديليت في الحوار دا
- و بيعيد عداد الايدينتيتي هتقولي يعني ايه يعني مثلا انت عندك بيبقا في بعض الداتا بيزيس عداد ID بيزيد تلقائي كل ما تدخل داتا ف لما تستخدم الترانكيت بترست العداد دا
- ومبتشتغلش مع التريجرز يعني ايه تريجير اصلا التريجر دا بيبقا كود كده بيشتغل مع الديليت والانسيرت و الابديت لما بنغير في الداتا بنخليه يعمل حاجة اوتوماتيك من غير ما نقعد نعملها احنا كانك بتديله تريجر انه لو حصل كذا نفذ كذا اوتوماتيك مثال علي كده مثلا انت لو جه اوردر هيتكرت

ريكورد جديد بالاوردر دا فتروح تعمل تريجر يخليه لما
الاوردر يتعمل يروح ينقص الانفينيتوري ب 1

```

Edit  Copy

CREATE TRIGGER trg_AfterInsertOrder
ON Orders
AFTER INSERT
AS
BEGIN
    UPDATE Inventory
    SET Quantity = Quantity - 1
    WHERE ProductID IN (SELECT ProductID FROM Inserted);
END;

```

● فالتريجر دا مبيشتغلش مع الترانيكيت

- انما الديلييت بقا ممكن تدلييت حاجة واحده او تدلييت كله من غير ما
تضيف شرط الوير او تدلييت مجموعه معينه بالوير وببشتغل معاها
التريجر عادي بس مبرستش ال ID اللي ببيقا اوتو انكريمينت دا

مقارنة سريعة				
الأمر	يُمسح بيانات	يُمسح الجدول	فيه شرط WHERE	تقدر تعمل ROLLBACK
DELETE	✓	✗	✓	✓
TRUNCATE	✓ (كلها)	✗	✗	أحيانًا (حسب DBMS)
DROP	✓	✓	✗	✗

رول باك دي انك تتراجع عنها عشان الديلييت مش اوتوكوميتيد ف عادي
تروول باك

3. Data Control Language

DCL commands are used to grant and take back authority from any database user.

Here are some commands that come under DCL:

- Grant
- Revoke

دي عشان تدي اكسس للناس علي الداتا بيز و علي الكوماندت

a. **Grant:** It is used to give user access privileges to a database.

Example

1. GRANT SELECT, UPDATE ON MY_TABLE TO SOME_USER, ANOTHER_USER;

b. **Revoke:** It is used to take back permissions from the user.

Example

1. REVOKE SELECT, UPDATE ON MY_TABLE FROM USER1, USER2;

جرانت بتدي اكسس و ريفوك بتسحب الاكسيس

4. Transaction Control Language

TCL commands can only use with DML commands like INSERT, DELETE and UPDATE only.

These operations are automatically committed in the database that's why they cannot be used while creating tables or dropping them.

Here are some commands that come under TCL:

- COMMIT
- ROLLBACK
- SAVEPOINT

دا بقا اوامر خاصه بموضوع الترانسأكشن دا و حوار المعاملات الماليه وكده

a. Commit: Commit command is used to save all the transactions to the database.

Syntax:

1. COMMIT;

Example:

1. DELETE FROM CUSTOMERS
2. WHERE AGE = 25;
3. COMMIT;

دا لما بيتكتب بعد ال DML بخليها تسمع في الداتا بيز عشان اصلا ال DML هي نون اوتو كوميتينج

b. **Rollback:** Rollback command is used to undo transactions that have not already been saved to the database.

Syntax:

1. ROLLBACK;

Example:

1. DELETE FROM CUSTOMERS
2. WHERE AGE = 25;
3. ROLLBACK;

رول باك دي عشان لو حصل مشكله اثناء تنفيذ عمليه ترجع الداتا بيز للنسخه القديمه عشان متضيفش داتا بايظه او مش صح او تمسح داتا فهي بتستخدم عشان ترجع الترانسأكشن اللي متسيفتش في الداتا بيز

c. **SAVEPOINT:** It is used to roll the transaction back to a certain point without rolling back the entire transaction.

Syntax:

1. SAVEPOINT SAVEPOINT_NAME;

عشان اعمل تشيك بوينت كده اقدر ارجلها في اي وقت بدل ما اني امسح عمليه الترانسأكشن كلها لا انا هرجع لنقطه معينه جواها

📌 ما هو TCL؟

TCL = Transaction Control Language

هي مجموعة أوامر في SQL مسؤولة عن إدارة المعاملات (Transactions) في قاعدة البيانات.

المعاملات معناها: مجموعة أوامر SQL تُنفَّذ كوحدة واحدة، بحيث يا إما تنجح كلها يا إما تفشل كلها (جزء من مبدأ ACID اللي شرحناه قبل).

♦ أوامر TCL الأساسية

فيه 4 أوامر رئيسية:

1. BEGIN / START TRANSACTION

2. COMMIT

3. ROLLBACK

4. SAVEPOINT

1 BEGIN / START TRANSACTION

- بيبدأ معاملة جديدة.
- بعده تكتب أوامر SQL (INSERT, UPDATE, DELETE) قبل ما تعمل COMMIT أو ROLLBACK.

مثال:

Edit Copy

sql

```
START TRANSACTION;
```

```
UPDATE Accounts
```

```
SET Balance = Balance - 100
```

```
WHERE AccountID = 1;
```

```
UPDATE Accounts
```

```
SET Balance = Balance + 100
```

```
WHERE AccountID = 2;
```

هنا لسه التغييرات مش محفوظة في الداتابيز بشكل دائم لحد ما تعمل COMMIT.

Saved memory full ⓘ

COMMIT 2

- يحفظ كل التغييرات التي تمت في المعاملة ويجعلها دائمة.
- بعد COMMIT ما تقدرش ترجع للبيانات القديمة.

مثال:

Edit Copy

sql

```
COMMIT;
```

بعد الكوميت، التحويل البنكي في المثال السابق أصبح نهائي.

ROLLBACK 3

- يرجع قاعدة البيانات لحالتها قبل بداية المعاملة.
- يستخدم لو حصل خطأ أو ما حبيت تحفظ التغييرات.

مثال:

Edit Copy

sql

```
ROLLBACK;
```

في مثال التحويل البنكي، لو خصمت المبلغ من الحساب الأول لكن ما قدرتش تضيفه للحساب الثاني بسبب خطأ، تعمل ROLLBACK علشان ترجع الفلوس.

SAVEPOINT 4

- بيعمل نقطة حفظ داخل المعاملة.
- تقدر تعمل ROLLBACK لحد نقطة معينة بدل ما ترجع بداية المعاملة كلها.

مثال:

Edit Copy

sql

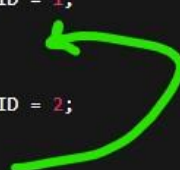
```
START TRANSACTION;
```

```
UPDATE Accounts SET Balance = Balance - 100 WHERE AccountID = 1;
```

```
SAVEPOINT after_withdraw;
```

```
UPDATE Accounts SET Balance = Balance + 100 WHERE AccountID = 2;
```

```
ROLLBACK TO after_withdraw; -- يرجع بعد خصم الفلوس فقط
```



سيناريو عملي شامل

تخيل إنك عندك برنامج إدارة متجر:

1. تدخل منتجات جديدة.
2. تحدّث الكميات.
3. تحذف منتج قديم.

لو كل العمليات دي جزء من معاملة واحدة:

Edit Copy

sql

```
START TRANSACTION;
```

```
INSERT INTO Products VALUES (101, 'Laptop', 5000, 10);
```

```
UPDATE Products SET Quantity = Quantity - 2 WHERE ProductID = 55;
```

```
DELETE FROM Products WHERE ProductID = 77;
```

```
-- لو كله تمام --
```

```
COMMIT;
```

```
-- لو فيه مشكلة --
```

```
-- ROLLBACK;
```

دا مثال وشرح حلو للترانسأكشن دا

نيجي بقا لحوار الاسيد دا يعني ايه اسيد الاسيد دا يزميلي مبادئ في الداتايبز لازم الداتايبز بتاعتك تحققها عشان تبقا ماشيه كويس وميحصلش مشاكل

تمام

مفهوم ACID هو مجموعة من الخصائص الأساسية التي لازم أي Database Transaction تحافظ عليها عشان تضمن سلامة البيانات، خصوصًا في قواعد البيانات العلائقية (Relational Databases).

معنى كلمة ACID

= ACID

- A → Atomicity
- C → Consistency
- I → Isolation
- D → Durability

Saved memory full ⓘ

1 Atomicity (الذرية)

المعنى:

المعاملة (Transaction) بتنفذ كلها أو ما بتنفذش خالص. لو أي خطوة فشلت، النظام بيرجع (Rollback) البيانات للحالة قبل المعاملة.

مثال:

تحويل 100 جنيه من حسابك لحساب صاحبك:

1. خصم 100 جنيه من حسابك.
2. إضافة 100 جنيه لحساب صاحبك.

لو حصل خطأ بعد الخطوة الأولى، بيرجع المبلغ لحسابك كأن العملية ما حصلتش.

2 Consistency (الاتساق)

المعنى:

بعد المعاملة، البيانات لازم تفضل في حالة سليمة وتراعي قواعد الداتايبز (Constraints, Triggers, Rules). أي Transaction لازم تحوّل البيانات من حالة سليمة إلى حالة سليمة أخرى.

مثال:

لو في قاعدة بتقول "الرصيد ما ينفعش يكون أقل من صفر"، ما ينفعش معاملة تسبب رصيد بالسالب.

3 Isolation (العزل)

📌 المعنى:

المعاملات (Transactions) تنفذ كأنها لوحدها، حتى لو فيه معاملات ثانية شغالة في نفس الوقت. العزل يمنع المشاكل زي Dirty Reads و Lost Updates.

💡 مثال:

لو أنا بسحب فلوس وأنت بتحول فلوس لنفس الحساب في نفس اللحظة، النظام يضمن إن العمليتين ما تلخبطوش بعض.

4 Durability (الدوام)

📌 المعنى:

بعد ما المعاملة تخلص وتعمل Commit، التغييرات تبقى محفوظة بشكل دائم—even لو السيرفر وقع أو الكهرباء قطعت.

💡 مثال:

لو حولت فلوس وبعدين الكهرباء قطعت، لما السيستم يرجع، العملية تكون موجودة في السجلات ومحفوظة.

لو مفهمتش اوي المثال دا هيفهمك

🎯 مثال عملي شامل

لو بتحجز تذكرة طيران:

1. Atomicity: إما يتم الحجز + خصم المبلغ معًا أو لا يتم أي منهما.
2. Consistency: ما يتفعلش يتجاوز عدد الحجزات عدد المقاعد المتاحة.
3. Isolation: لو فيه شخصين بيعجزوا آخر مقعد، واحد بس هو اللي يكسبه.
4. Durability: بعد الحجز، العملية محفوظة حتى لو السيستم وقع بعد ثانية.

5. Data Query Language

DQL is used to fetch the data from the database.

It uses only one command:

○ SELECT

a. SELECT: This is the same as the projection operation of relational algebra. It is used to select the attribute based on the condition described by WHERE clause.

Syntax:

1. SELECT expressions
2. FROM TABLES
3. WHERE conditions;

For example:

1. SELECT emp_name
2. FROM employee
3. WHERE age > 20;

Edit  Copy 

sql

```
SELECT [DISTINCT] column1, column2, ...  
FROM table_name  
[WHERE condition]  
[GROUP BY column]  
[HAVING condition]  
[ORDER BY column [ASC|DESC]]  
[LIMIT number]; -- (MySQL / PostgreSQL)
```

◆ شرح الأجزاء:

1. **SELECT** → تحدد الأعمدة اللي عايز تجيبها.
 - `SELECT Name, Age`
 - أو كل الأعمدة: `* SELECT`
2. **FROM** → تحدد الجدول اللي هتسحب منه البيانات.
3. **WHERE** → فلترة الصفوف بشرط.
 - `WHERE Age > 20`
4. **DISTINCT** → تمنع التكرار.
 - `SELECT DISTINCT DeptID`
5. **ORDER BY** → ترتيب النتائج.
 - `ORDER BY Age DESC`
6. **GROUP BY** → تجميع حسب عمود.
 - `GROUP BY DeptID`
7. **HAVING** → شرط على نتائج الـ GROUP BY.
 - `HAVING COUNT(*) > 2`
8. **LIMIT** → تحدد عدد الصفوف اللي ترجع (MySQL/PostgreSQL).
 - `LIMIT 5`

بس شوفلك بقا اي شيت شيت لل SQL

أمثلة SELECT متدرجة

1 اختيار كل الأعمدة

Edit ✎ Copy 📄

sql

```
SELECT *  
FROM Students;
```

2 اختيار أعمدة معينة + استخدام AS لإعادة تسمية الأعمدة

Edit ✎ Copy 📄

sql

```
SELECT Name AS StudentName, Age AS StudentAge  
FROM Students;
```

3 استخدام WHERE مع شرط

Edit ✎ Copy 📄

sql

```
SELECT Name, Age  
FROM Students  
WHERE Age > 21;
```

Saved memory full ⓘ

4 استخدام LIKE للبحث بالنصوص

Edit ↗ Copy ↗

sql

```
SELECT Name
FROM Students
WHERE Name LIKE 'A%'; -- يبدأ بحرف A
```

5 استخدام ORDER BY

Edit ↗ Copy ↗

sql

```
SELECT Name, Age
FROM Students
ORDER BY Age DESC;
```

6 استخدام DISTINCT

Edit ↗ Copy ↗

sql

```
SELECT DISTINCT DeptID
FROM Students;
```

Saved memory full ⓘ

GROUP BY + HAVING استخدام 7

Edit ↗ Copy 📄

sql

```
SELECT DeptID, COUNT(*) AS NumStudents
FROM Students
GROUP BY DeptID
HAVING COUNT(*) >= 2;
```

INNER JOIN 8

Edit ↗ Copy 📄

sql

```
SELECT S.Name, D.DeptName
FROM Students AS S
INNER JOIN Departments AS D
ON S.DeptID = D.DeptID;
```

LEFT JOIN 9

Edit ↗ Copy 📄

sql

```
SELECT S.Name, D.DeptName
FROM Students AS S
LEFT JOIN Departments AS D
ON S.DeptID = D.DeptID;
```

Saved memory full ⓘ

RIGHT JOIN 10

Edit ✎ Copy 📄

sql

```
SELECT S.Name, D.DeptName
FROM Students AS S
RIGHT JOIN Departments AS D
ON S.DeptID = D.DeptID;
```

➔ يجب كل الأقسام حتى لو مفياش طلاب.

FULL OUTER JOIN (لو ال DBMS بيدعمها) 1 1

Edit ✎ Copy 📄

sql

```
SELECT S.Name, D.DeptName
FROM Students AS S
FULL OUTER JOIN Departments AS D
ON S.DeptID = D.DeptID;
```

استخدام IN 1 2

Edit ✎ Copy 📄

sql

```
SELECT Name
FROM Students
WHERE DeptID IN (1, 2);
```

Saved memory full ⓘ

استخدام ANY 1 3

Edit ↗ Copy 📄

sql

```
SELECT Name, Age
FROM Students
WHERE Age > ANY (SELECT Age FROM Students WHERE DeptID = 1);
```

➔ يجيب الطلاب اللي عمرهم أكبر من أي طالب في قسم 1.

استخدام ALL 1 4

Edit ↗ Copy 📄

sql

```
SELECT Name, Age
FROM Students
WHERE Age > ALL (SELECT Age FROM Students WHERE DeptID = 1);
```

➔ يجيب الطلاب اللي عمرهم أكبر من كل الطلاب في قسم 1.

استخدام BETWEEN 1 5

Edit ↗ Copy 📄

sql

```
SELECT Name, Age
FROM Students
WHERE Age BETWEEN 20 AND 22;
```

استخدام IS NULL 1 6

Edit Copy

sql

```
SELECT Name
FROM Students
WHERE DeptID IS NULL;
```

استخدام LIMIT 1 7

Edit Copy

sql

```
SELECT *
FROM Students
LIMIT 2;
```

أول صفين فقط (MySQL/PostgreSQL) ➔

مثال شامل (فيه كل حاجة تقريباً) 🔥

Edit Copy

sql

```
SELECT S.Name AS StudentName, S.Age, D.DeptName
FROM Students AS S
INNER JOIN Departments AS D
ON S.DeptID = D.DeptID
WHERE S.Age > ANY (SELECT Age FROM Students WHERE DeptID = 1)
AND S.Name LIKE '%a%'
GROUP BY S.Name, S.Age, D.DeptName
HAVING COUNT(*) >= 1
ORDER BY S.Age DESC
LIMIT 5;
```